

```
In [18]: import pandas as pd
import numpy as np
from math import log2
import json
```

```
In [19]: df = pd.read_csv("data/play_tennis.csv")
print("CSV file read!")
```

CSV file read!

```
In [20]: def entropy(target_col):
    elements, counts = np.unique(target_col, return_counts=True)
    entropy_value = 0
    for i in range(len(elements)):
        p = counts[i] / np.sum(counts)
        entropy_value += -p * log2(p)
    return entropy_value
```

```
In [21]: def info_gain(data, split_attribute, target_name="PlayTennis"):
    total_entropy = entropy(data[target_name])

    values, counts = np.unique(data[split_attribute], return_counts=True)

    weighted_entropy = 0
    for i in range(len(values)):
        subset = data[data[split_attribute] == values[i]]
        weighted_entropy += (counts[i] / np.sum(counts)) * entropy(subset[target_name])

    return total_entropy - weighted_entropy
```

```
In [22]: def id3(data, original_data, features, target="PlayTennis", parent_node_class=None):

    # If all target values same → return that class
    if len(np.unique(data[target])) <= 1:
        return np.unique(data[target])[0]

    # If dataset empty → return most common class
    elif len(data) == 0:
        return np.unique(original_data[target])[np.argmax(
            np.unique(original_data[target], return_counts=True)[1])]

    # If no features left → return parent node class
    elif len(features) == 0:
        return parent_node_class

    else:
        parent_node_class = np.unique(data[target])[np.argmax(
            np.unique(data[target], return_counts=True)[1])]

        # Select best feature
        item_values = [info_gain(data, feature, target) for feature in features]
        best_feature_index = np.argmax(item_values)
        best_feature = features[best_feature_index]
```

```

tree = {best_feature: {}}

features = [i for i in features if i != best_feature]

for value in np.unique(data[best_feature]):
    subset = data[data[best_feature] == value]
    subtree = id3(subset, data, features, target, parent_node_class)
    tree[best_feature][value] = subtree

return tree

```

```

In [24]: features = df.columns[:-1].tolist()
         tree = id3(df, df, features)

         print(json.dumps(tree, indent=2))

```

```

{
  "Outlook": {
    "Overcast": "Yes",
    "Rain": {
      "Wind": {
        "Strong": "No",
        "Weak": "Yes"
      }
    },
    "Sunny": "No"
  }
}

```

In []: